Project Contract

WG Management System (WGMS)

Software Engineering Project – Branching Pipeline Methodology

Team Members: Cemil Aslan, Dzovani Koller, Joan de la Fuente Camps,
Daniel Petriceanu, Taras Trishch, Andrei Ciuta

June 1, 2026

Preamble

This Project Contract serves as the formal technical commitment to realize the **Value Proposition** established in the Project Proposal. Its primary goal is to address the identified sponsor pain points through the specific software functions and architectural assets defined below.

Methodology and Scope Note: The project uses the **Branching Pipeline Methodology**. The project deliberately concentrates on a small set of well-tested core features: registration, login and WG management with role-based access control; task creation, assignment, tracking and recurring scheduling; well-tested shared expense tracking and settlement; collaborative shopping management with structured items and shopping history; and a central dashboard with a statistical history view. Features such as NFC appliance logging, voice automation, LLM-based command parsing, consumable stock prediction, and cross-WG event/communication tooling are **explicitly out of scope**. Planned extended features include a swipe-based settlement-approval interface (demonstrated with a dedicated test account) and a QR-code scanner for one-click shopping item entry.

Contents

1 Team Members & Roles **3**

2 Project Description **3**
2.1 Initial Situation & Existing Similar Products . . . 3
2.2 Project Overview & Technical SE Goals . . . 6
2.3 Target Audience . . . 7

3 Asset Architecture & System Environment **9**
3.1 Asset Network & Infrastructure Stack . . . 9
3.2 Gap & Risk Analysis . . . 10
3.3 C4 Model Architecture . . . 10

4 Requirements **12**
4.1 Functional Requirements (Use Cases) . . . 12
4.2 Scope and Priority Policy . . . 12
4.3 Non-Functional Requirements . . . 13
4.4 Supported Platforms and Optimization Targets . . . 14
4.5 Iceberg List (Features & Estimation) . . . 14

5 Domain Model **15**
5.1 Domain Model Explanation . . . 17

6 Working Structure & Project Plan **17**
6.1 Distribution of Roles . . . 17
6.2 Horizontal Responsibilities . . . 17
6.3 Project Phases & Milestones . . . 18
6.4 Project Boundaries (Out-of-Scope) . . . 18

7 Supply Components (Deliverables) **19**

8 Risk Assessment and Quality Management **20**
8.1 Risk Matrix . . . 20
8.2 Quality Assurance Strategy . . . 22
8.3 Information Process . . . 22

9 Legal, Compliance and External Hosting Constraints **24**
9.1 Processed Data Categories . . . 24
9.2 GDPR-Oriented Design Principles . . . 24
9.3 User Rights and Data Handling . . . 24
9.4 External Hosting Constraints . . . 25
9.5 Security Constraints . . . 25
9.6 Residual Compliance Risk . . . 25

1 Team Members & Roles

Name	Student ID	Primary Role	Backup Role
Cemil Aslan	12423834	Team Coordinator	Technical Architect
Dzovani Koller	12432196	Technical Architect	Fullstack Developer
Joan de la Fuente Camps	12038098	UI/UX	Technical Architect
Daniel Petriceanu	12320180	Documentation Lead	UI/UX
Taras Trishch	12229192	DevOps	Technical Architect
Andrei Ciuta	12326215	Test Lead	Team Coordinator

2 Project Description

2.1 Initial Situation & Existing Similar Products

Initial Situation and Sponsor Pain Points

A typical shared flat (*Wohngemeinschaft*, WG) in Austria or Germany operates as a small informal organisation: rent and utilities are pooled, groceries are bought ad hoc by whoever is at the supermarket, chores rotate by spoken or hand-written agreement, and the financial liability for the household is concentrated on a single main tenant (*Hauptmieter*in*) who holds the lease. The tooling that supports this organisation, however, is fragmented across at least four parallel channels: a WhatsApp group for communication, a printed schedule on the fridge for chores, an expense-splitting app such as Splitwise for receipts, and the memory of the main tenant for everything else. Receipts disappear into chat history, paper schedules decay within weeks, and verbal agreements are forgotten between the people who made them and the people who join the household later.

The recurring failure modes of this arrangement constitute the sponsor pain points the WGMS project commits to address:

- **Financial leakage on the main tenant.** Micro-expenses below approximately five euros are systematically not reclaimed because the social cost of asking exceeds the monetary value. Over an eleven-month academic year, the cumulative under-recovery is non-trivial and is borne entirely by the lease holder.
- **Unbounded coordination overhead.** Monthly *Kassensturz* (manual reconciliation) requires the main tenant to scroll weeks of WhatsApp history, cross-reference receipts, and individually chase three to five roommates. The opportunity cost is several hours of work per month that produces no output beyond restoring an accurate balance.
- **Liability without authority.** The main tenant carries legal responsibility toward the landlord and economic responsibility toward third parties, but has no enforceable mechanism to ensure that roommates discharge their share. Disputes therefore escalate to the level of personal relationships rather than being resolved at the tooling layer.
- **Onboarding gap on roommate turnover.** When a roommate joins or leaves mid-term, the verbal and paper-based record of the household state cannot be transferred. The new roommate inherits an opaque set of conventions and is routinely surprised by their first overdue chore or first unexpected expense, eroding goodwill from day one.

Existing Similar Products

Three categories of solution currently address fragments of the problem above. None addresses it as a whole.

Splitwise is the dominant English-language expense-splitting service, available on the web, iOS, and Android. Its strength is a mature settlement model that handles unequal splits, multiple currencies, and partial payments. With respect to the WG use case it is purely a financial ledger: it has no concept of recurring chores, shopping lists, household membership, or shared communication. WGs that adopt Splitwise must continue to maintain WhatsApp, paper schedules, and any shopping coordination outside the tool.

Flatastic (also marketed as *Die WG-App*) is the closest direct competitor in the German-speaking market. It is a mobile-first application that bundles four modules: a cleaning schedule with a points-based ranking system, a synchronised shopping list, a shared expense ledger, and a pinboard called *Shouts* for lightweight intra-WG communication. Flatastic is offered as a freemium product with a subscription that unlocks features such as expense export, photo attachments on receipts, and per-item shopping notes. Flatastic has a substantial installed base and a long product history, but is constrained in three ways relevant to WGMS. First, its primary surface is mobile, which excludes desktop-first administrative workflows. Second, it treats every household uniformly — there is no domain-level distinction between a financially-liable main tenant and an ordinary roommate. Third, it has no concept of inter-WG interaction.

Splid represents a third class of competitor: lightweight, single-purpose group expense trackers (alongside Tricount and similar apps). These are excellent for tracking one trip or one project but do not provide the persistent, role-aware household model that a long-lived WG requires.

Bring! is a dedicated collaborative shopping list application widely used in German-speaking households. It offers a structured item catalogue with autocomplete and popular item suggestions, which removes the need to type free-text item names repeatedly. WGMS draws on this approach for its shopping module: items are treated as structured catalogue entries rather than plain strings, with autocomplete and common item suggestions where feasible.

Status quo: WhatsApp and paper. The actual incumbent that WGMS must displace is the combination of a WhatsApp group, a fridge-mounted cleaning schedule, and the ad-hoc memory of whoever paid last. This combination has the lowest possible onboarding cost (zero) and the highest possible familiarity. Any specialised tool must demonstrably outperform it on the workflows that matter most — adding an expense, viewing the current balance, and confirming that a chore is done — to be adopted at all.

Identified Gap and Unique Selling Propositions

Table 1 identifies a concrete gap: existing products cover individual aspects of shared household management, but they do not provide a role-aware integrated household system with a strong focus on coordination and financial transparency.

WGMS addresses this through five main unique selling propositions:

- **Single source of truth for WG management.** Tasks, expenses, and shopping activities are managed in one unified system instead of fragmented tools.
- **Role-aware household model.** WGMS distinguishes between Main Tenant and Roommate roles through backend-enforced Role-Based Access Control. Administrative actions such as invitation handling and household management are restricted to authorized users.

Capability	WhatsApp + paper	Splitwise	Splid	Flatastic	WGMS
Shared expense tracking	–	✓	✓	✓	✓
Settlement engine	–	✓	partial	✓	✓
Recurring task scheduling	paper	–	–	✓	✓
Shared shopping list	–	–	–	✓	✓
Targeted shopping requests	–	–	–	–	✓
QR-assisted one-click shopping	–	–	–	–	✓
Persistent household roles	–	–	–	–	✓
Invitation onboarding	–	–	–	partial	✓
Statistical dashboard view	–	partial	–	partial	✓
Web-first access	–	✓	–	partial	✓
GDPR-aligned design	n/a	partial	partial	✓	✓

Table 1: Feature coverage of WGMS against the most relevant existing solutions. *Partial* indicates that the capability exists in a degraded or limited form (e.g. a chore module without a statistical history view).

- **Deterministic settlement engine.** Shared expenses and debt balancing are calculated using `java.math.BigDecimal` with fixed rounding behaviour. Settlement logic is isolated in dedicated services and verified through unit tests.
- **Collaborative shopping with structured items.** Shopping workflows use a structured item catalogue, support per-item visibility (personal, targeted, or shared), and maintain a transparent shopping history. A QR-code scanner for one-click item entry is planned as a later extended feature.
- **Historical dashboard and transparency.** The dashboard provides insight into spending behaviour, purchase history, category distributions, and user contributions using statistical visualizations rather than predictive models.

Business Model and Monetisation Paths

The Project Contract scope does not include payment processing, ad serving, or B2B contracting. The architecture is, however, designed to be compatible with the following commercial paths, recorded here to motivate design decisions (the role model, the GDPR-aligned data handling) without expanding the implementation scope. None of these paths produces a deliverable in this contract or influences grading.

- **Freemium with non-invasive advertising.** The core workflows — registration, WG creation, expenses, chores, shopping, dashboard — remain free and ad-supported through placements modelled on the New York Times Games approach: a single static banner per session, no interstitials, no interruption of task flows. A small subset of quality-of-life features (e.g. data export, advanced filters) may later move behind an ad-free subscription. The workflows that drive adoption are never paywalled, ensuring parity with the zero-cost incumbent (WhatsApp + paper).
- **Micro-fee on settlement transactions.** A small fixed fee in the order of one to five euro cents per recorded settlement could be levied to monetise the deterministic settlement engine without burdening micro-expenses themselves. The fee would be capped per month and waived in the freemium tier above a configurable threshold, so users who settle once a month pay nothing measurable.

- **B2B partnership with property management firms (Immobilienverwaltungen).** Property management firms that lease student housing inventory absorb the indirect cost of tenant disputes through support overhead, escalations to the landlord, and reduced retention. A licensing arrangement allows such firms to offer WGMS as a pre-installed amenity for their tenants, with the firm paying a per-WG fee and the residents receiving the ad-free experience by default. A future portfolio role for the firm would be a strictly read-only aggregate view across its WGs — no leakage between flats.
- **Student-targeted sponsorships.** A WG is, in aggregate, a small commercial cohort with a predictable life cycle. Targeted sponsorship slots could be offered to partners whose products fit that cohort: banks promoting student accounts, local event services such as Goodnight.at or Freyzeit.at, and the kind of merchant offers found in printed *Gutscheinhefte*. Sponsorship slots would be served through the same banner placements as the freemium tier, never inside task flows, and would be configurable per WG so that households can opt out of categories they find irrelevant.

All four paths are referenced in Section 6.4 as commercial options that the system is designed to *not preclude*. The role-aware data model, the household-scoped privacy boundary, and the GDPR-aligned data handling are the technical preconditions for any of them.

2.2 Project Overview & Technical SE Goals

Project Overview

WGMS is a responsive browser-based information system that consolidates the operational management of a shared household into a single application. The system is implemented as an Angular Single Page Application backed by a Java Spring Boot REST API and an embedded H2 database.

Functionally, the system is organized around five core use case packages:

- User registration, login and WG initialization
- Task scheduling and recurring household management
- Shared expense tracking and settlement
- Collaborative shopping management with shopping history
- Dashboard with a statistical history view

The application intentionally focuses on practical and testable functionality instead of experimental features. The system processes household-related information such as user profiles, task records, shopping history, and expense data, but does not execute monetary transactions or process sensitive GDPR categories.

Technical Software Engineering Goals

The Project Contract commits the team to the following measurable technical goals. Each goal is phrased so that fulfilment can be assessed directly from the GitLab repository and the deployed artefact, in line with the assessment criteria communicated by the SEPR teaching team.

- **TG-01 — Three-tier architecture.** A clean separation between Angular SPA, Spring Boot REST API, and persistence layer (Spring Data JPA over H2). No business logic in the frontend; no database access outside repositories.

- **TG-02 — Authentication and authorisation.** Stateless authentication via JSON Web Tokens, password hashing via BCrypt, and role-based access control enforced on the backend (not only hidden in the frontend). Two roles are distinguished at the domain level: *Main Tenant* and *Roommate*.
- **TG-03 — Deterministic financial logic.** The settlement engine reduces the inter-roommate debt graph to its minimum-transactions form using `java.math.BigDecimal` with HALF_UP rounding to two decimal places, executed within a Spring `@Transactional` service boundary. Settlement correctness is verified by unit tests covering equal splits, unequal splits, rounding edge cases, deleted-user fallbacks, and settlement status after recorded payments.
- **TG-04 — API documentation and contracts.** All REST endpoints are described by an OpenAPI (Swagger) specification served by the running application. The Angular client consumes this contract, and integration tests verify that documented and actual behaviour agree.
- **TG-05 — Continuous integration and deployment.** Every push to a feature branch triggers a GitLab pipeline that builds the project, runs the full test suite, and (on the main branch) builds a Docker image, pushes it to the project registry, and deploys it to the university Kubernetes cluster. No deployment artefact is produced manually.
- **TG-06 — Statistical dashboard implementation.** The system shall provide historical visualizations including expense categories (e.g. operating costs, food, electricity), spending development over time, shopping history, and user contribution statistics (who bought most). The implementation is verified through frontend and integration tests using generated test data.
- **TG-07 — GDPR-aligned data handling.** The system supports account deletion or anonymization, time-limited invitation links, and documented handling of stored user data, aligned with the GDPR checklist referenced in NFR-02.
- **TG-08 — Browser support.** The application targets current versions of Chrome, Firefox, Safari, and Edge and is verified through cross-browser smoke tests.
- **TG-09 — Critical logic test coverage.** Authentication, invitation handling, task scheduling, and settlement logic are covered by automated unit, repository, and integration tests.
- **TG-10 — Maintainability and documentation.** The repository includes architecture documentation, user documentation, API documentation, and setup instructions.

The main engineering contributions of the project are the deterministic settlement engine, role-based WG management, recurring task scheduling, collaborative shopping, and historical dashboard visualizations.

2.3 Target Audience

The product is aimed at two primary user roles, plus a sponsor and one secondary audience relevant to architectural decisions but not implemented in the version covered by this contract.

Project Sponsor

For the purpose of the SEPR course, the Project Sponsor is the teaching team of the *Software Engineering Project* module at TU Wien, represented by the assigned assistant. The sponsor's acceptance criterion is the fulfilment of this Project Contract: the implementation of the high- and medium-priority features in Section 4.5, the architectural commitments in Section 3, and

the quality and risk commitments in Sections 8 and 9. Grading is based on the contents of the GitLab repository at each Milestone Review (MR1–MR3) and on the demonstrated working software.

In a hypothetical commercial deployment of the same system the *de facto* sponsor would be the Main Tenant role described below, who carries the financial liability of the household, derives the largest share of value from the tool, and is the role most likely to introduce WGMS into a flat and to advocate for its adoption by the other roommates.

Persona 1: The Main Tenant

Name and role. Anna Berger, 26, master's student in Business Informatics at TU Wien, main tenant of a four-person WG in the 9th district of Vienna for the past two years.

Context. Anna signed the rental contract with the landlord and pays the consolidated rent, electricity, and internet bills from her account. Her three roommates transfer their shares back to her each month. Two of them are reliable; one is chronically late.

Goals. Recover every euro that the WG owes her without an awkward conversation. Spend less than thirty minutes per month on household administration. Hand over an accurate record to a new roommate when one moves out.

Frustrations. Receipts that disappear into the WhatsApp group. The cumulative cost of three- and four-euro items she has stopped asking about. The end-of-month *Kassensturz*, which currently consumes between three and five hours. Awkwardness when she has to "remind" a friend who is also a roommate about a debt.

Quote. "I shouldn't have to be the WG's accountant just because my name is on the lease."

How WGMS helps Anna. The dashboard surfaces every open balance and overdue task at a glance (FR-12) and the statistical history view shows how spending develops over time (FR-16). The settlement engine resolves debts deterministically (FR-05, FR-06). Reminders (FR-11) replace the awkward in-person conversation. The history view (FR-13) gives her a defensible audit trail when a roommate disputes a charge.

Persona 2: The Roommate

Name and role. Lukas Stein, 22, third-year bachelor's student in Computer Science, joined Anna's WG six months ago after the previous tenant moved out.

Context. Lukas moved into an existing household mid-semester. The cleaning rotation, the shared shopping conventions, and the previous balance owed to the main tenant were all communicated to him verbally on move-in day. He has retained roughly half of it.

Goals. Pull his weight without having to ask repeatedly what is expected. Pay his share of expenses promptly so that he is not the one being chased. Avoid being the WG member that others complain about behind his back.

Frustrations. The WhatsApp group has more than a thousand messages from before he joined and is impossible to catch up on. He missed the first cleaning rotation because nobody told him about it. He has paid for groceries twice and forgotten to claim the second one.

Quote. "I'm happy to pay my share — I just need someone to actually tell me what it is."

How WGMS helps Lukas. On joining the WG via a time-limited invitation link (FR-02), Lukas immediately sees his current share of open expenses, his assigned chores, and the active

shopping list (FR-12). Reminders (FR-11) flag a new task or expense. The one-click expense generation (FR-14) and the shared shopping list (FR-07) make claiming a shared purchase frictionless enough that he actually does it.

Secondary Audience

One further audience is not a user-facing role in the version of WGMS covered by this contract, but it motivates design choices that the implementation honours from the first milestone onward.

Property Management Firms. Firms that lease student housing inventory hold an indirect stake in the disputes WGMS aims to reduce: tenant escalations, landlord complaints, and turnover all carry support cost. No property-management role is implemented in this version, but the role-aware data model and the GDPR-aligned data handling (Section 9) make a future strictly read-only portfolio role technically feasible without data-model migration. This is the audience that motivates the B2B commercial path described in Section 2.1, and the design constraint it imposes — strict isolation between WGs by default — is honoured throughout the implementation.

3 Asset Architecture & System Environment

An asset is defined as any component, framework, or infrastructure that contributes value to the information system. The WGMS architecture follows the university baseline and extends it with domain-specific modules for user management, household management, task scheduling, shopping management, expense tracking, and dashboard analytics.

3.1 Asset Network & Infrastructure Stack

To fulfill the domain requirements while adhering to the technical directives, the project uses the following technology stack:

- **Client-Side (Frontend):** The system is implemented as a responsive Single Page Application (SPA) using **Angular 21.x**. Node.js **24.x LTS** is used for package management, development tooling, and production builds. The UI is optimized for the defined reference devices and browser versions listed in the compatibility section. The goal is not pixel-perfect support for every device, but a stable and usable experience on the agreed target platforms.
- **Server-Side (Backend):** The backend is a **Java Spring Boot 4.0.x** REST API targeting **OpenJDK 26** in the reference setup. It uses *Spring Security* for Role-Based Access Control (RBAC) and authentication via JSON Web Tokens. It also uses *MapStruct* for DTO mapping and *OpenAPI/Swagger* for interactive API documentation.
- **Data Persistence:** For the university project scope, the system uses an embedded **H2 Database 2.4.x** in automatic mixed mode. Data access is abstracted using **Spring Data JPA** and the Jakarta Persistence API (JPA). In deployed demo environments, the database file should either be stored on a persistent volume or recreated through documented seed data. WGMS does not claim production-grade database durability with embedded H2. A migration path to PostgreSQL is documented as future work.
- **Infrastructure & CI/CD:** The project uses **GitLab CI/CD**. After a successful pipeline run, the deployment job builds a Docker image, pushes it to the project registry, and deploys it to the university's **Kubernetes Cluster** where applicable. If the deployment environment

is limited, the minimum deliverable is a reproducible local Docker setup and documented deployment instructions.

3.2 Gap & Risk Analysis

Because WGMS extends the university baseline system with household-specific functionality, several implementation risks exist.

The first critical area is the **Settlement Engine**. Financial calculations require deterministic behaviour and precise rounding. This risk is mitigated through isolated business logic, transaction boundaries, and automated tests.

The second critical area is **Invitation and Role Management**. Incorrect permission handling may expose household information or allow unauthorized modifications. Backend-enforced RBAC and security tests mitigate this risk.

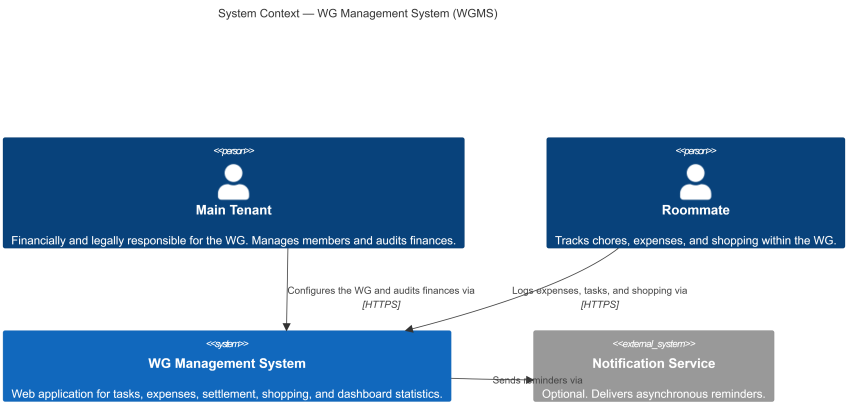
The third critical area is the **H2 Database Lifecycle**. Data persistence across deployments and restarts must be documented carefully. Seed data and migration scripts reduce this risk.

The fourth critical area is **Shopping Item Visibility**. Personal, targeted, and shared shopping items must be scoped correctly so that users see only items intended for them while shared items remain visible to the whole WG.

3.3 C4 Model Architecture

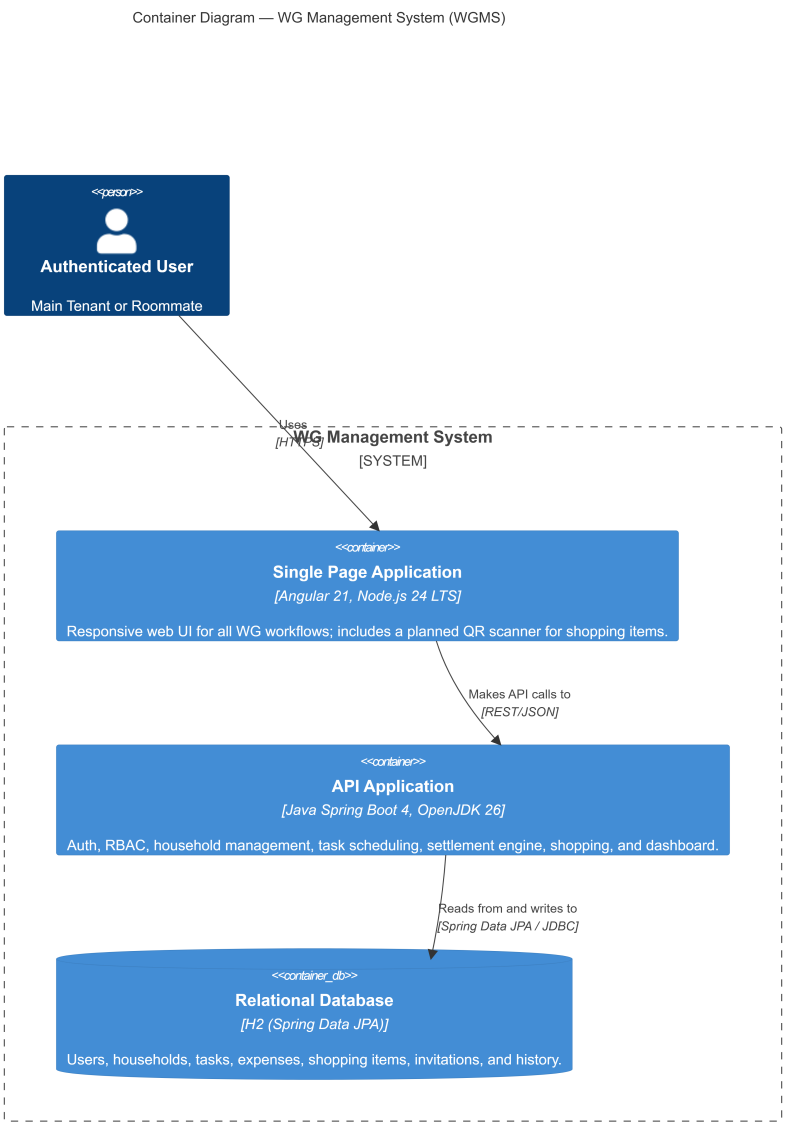
To visually bridge business requirements and technical implementation, the architecture is modeled using the C4 approach.

Level C1: System Context
The context diagram illustrates WGMS within its broader environment. The core actors are the *Main Tenant* and the *Roommate*. External interactions are limited to authentication and, where applicable, a notification service for asynchronous reminders.



Level C2: Container Diagram
Zooming into the system boundary, WGMS decomposes into the following containers:

- **Angular Web Application:** Delivers the responsive frontend interface and user workflows, including the planned QR scanner for shopping items.
- **Spring Boot API Service:** Contains authentication, authorization, household management, task scheduling, settlement logic, shopping management, and dashboard functionality.
- **H2 Database:** Stores users, households, tasks, expenses, shopping items, expense splits, payments, invitation tokens, and historical records.



4 Requirements

4.1 Functional Requirements (Use Cases)

The functional scope of the WG Management System (WGMS) contains the following core use case packages. The use case numbering follows the scope agreed with the teaching team. The packages previously covering cross-WG communication and event planning, the NFC appliance log, and voice/LLM automation have been removed from scope. Cross-WG communication, event planning, public feeds, and following other WGs are not part of this contract.

1. **UC-01: User Registration, Login and WG Initialization** (Priority: High)
   Allows users to register securely and log in. A user can create a new shared household (WG) and invite roommates through time-limited invitation links. Invitations are issued as single-use, securely hashed token links with a configurable expiry; the link is generated in-app and shared by the Main Tenant through their own channel. The current design does not dispatch invitation emails automatically, so no email-based password-reset flow is required. Role-based access control distinguishes the Main Tenant from ordinary Roommates.
2. **UC-02: Cleaning Schedule and Task Management** (Priority: High)
   Roommates can create, assign, and track household tasks and cleaning duties. Tasks support assignment to a roommate, filtering by status, undoing a completion in case of mistakes, and recurring scheduling so that chores repeat on a defined interval.
3. **UC-03: Shared Expense Tracking and Settlement** (Priority: High)
   Allows roommates to enter shared expenses, split costs, track balances, record payments between roommates, and monitor pending versus settled amounts. Settlement is deterministic and thoroughly tested. As an extended feature, a swipe-based settlement-approval interface (demonstrated with a dedicated test account) lets users quickly approve or reject pending settlements.
4. **UC-04: Collaborative Shopping Management** (Priority: High)
   Enables users to add personal or shared shopping requests, which can be aggregated into a household shopping list and kept as a shopping history. Shopping items are treated as structured catalogue entries with autocomplete suggestions for commonly used items, and support per-item visibility (personal, targeted, or shared). Items can be marked and unmarked as bought. As a later extended feature, a QR-code scanner will allow one-click addition of shopping items.
5. **UC-05: Central Dashboard and Statistical History** (Priority: High)
   Provides an aggregated dashboard showing open responsibilities, payment status, and active reminders, together with a **statistical history view**. Instead of predictive stock forecasting, the view presents pie charts and time series of operating costs, food and electricity spending, who bought the most, and how expenses develop over time.

4.2 Scope and Priority Policy

To keep the project feasible and well-tested within the available team effort, the project follows a strict priority policy.

Core functionality is prioritized before extended functionality is implemented. The core scope consists of registration, login, WG creation and invitations, task and cleaning schedule management with recurrence, shared expense tracking and settlement, collaborative shopping man-

agement with shopping history, dashboard functionality with a statistical history view, security basics, and documentation.

Extended functionality consists of a small, deliberately limited set: the swipe-based settlement-approval interface (with a test account), one-click expense generation, shopping item suggestion/voting, fair chore balancing via history, and a QR-code scanner for one-click shopping item entry. These features are implemented only after the core workflow is stable and well tested. If time is limited, extended features may be delivered as simplified prototypes or documented as future work.

The system is considered successful if the core WG-management workflow is functional, tested, documented, and demonstrable on the defined reference devices. Extended features improve the final result but should not endanger the delivery of the core system.

4.3 Non-Functional Requirements

In addition to the functional requirements, WGMS is designed to satisfy the following non-functional requirements. Each NFR names a concrete metric or checklist against which it is verified, in line with the feedback from the teaching team.

ID	Category	Requirement	Verification
NFR-01	Security & Access Control	Users must authenticate before accessing WG-specific data; passwords must be stored only as secure salted hashes; and access to sensitive functions must depend on the user's role (only the Main Tenant may manage invitations, remove members, or change important WG settings). The applicable security level is defined by the **OWASP Application Security Verification Standard (ASVS)** (owasp.org/.../asvs).	OWASP ASVS checklist review, role-based access tests, login tests
NFR-02	Data Privacy & GDPR	The system collects only data required for WG management and lets users access, correct, and delete or anonymize their account data where technically and legally possible. Compliance is measured against the **GDPR checklist** (gdpr.eu/checklist).	GDPR checklist review, manual compliance test cases
NFR-03	Reliability	Core data such as expenses, payments, tasks, and WG membership is persisted reliably through the database layer. Database initialization and seed data are documented and reproducible so that a demo environment can be restored to a known state.	Integration tests, documented seed-data restore

NFR-04	Usability & Compatibility	Main workflows (adding an expense, checking assigned tasks, viewing open payments) are completable without external instructions by a peer tester, and remain usable on the defined reference platforms. Usability and accessibility are checked against the **W3C WAI** guidance (w3.org/WAI) and/or a **Lighthouse** audit.	WAI checklist / Lighthouse audit, peer testing, reference-device tests
NFR-05	Maintainability	The codebase keeps a clear separation between frontend, backend, database access, and business logic. Important modules are documented sufficiently for handover, hand in hand with the user documentation.	Code review, architecture review, documentation review
NFR-06	Auditability	Important actions (creating expenses, settling payments, assigning tasks, changing WG membership) are traceable through timestamps and the responsible user. The audit metric is coverage of these defined critical actions by an audit log entry.	Audit-log inspection, functional tests

4.4 Supported Platforms and Optimization Targets

To avoid uncontrolled optimization effort, WGMS defines a limited set of reference platforms. Other devices are supported only on a best-effort basis and are not guaranteed to be pixel-perfect.

Category	**Reference Device**	**OS / Browser**	**Target Viewport**	**Support Level**
Primary iOS Mobile	iPhone 12 Pro	iOS 26.x / Safari	390 $\times$ 844 CSS px	Optimized and tested
Primary Android Mobile	Google Pixel 7	Android 16 / Chrome 147.x Stable or newer	412 $\times$ 915 CSS px	Optimized and tested
Small Mobile Boundary	Generic smartphone	Modern iOS/Android browser	Minimum 360 CSS px width	Should remain usable
Desktop Reference	Standard laptop display	Chrome 147.x or Firefox Stable	1366 $\times$ 768 px and higher	Optimized and tested

Tablets, foldables, smart TVs, and unusual embedded browsers are outside the dedicated optimization scope and are supported on a best-effort basis only.

4.5 Iceberg List (Features & Estimation)

The following table maps planned features to use cases, priorities, planned milestone versions, effort estimates, and responsible feature leads. The overall project effort budget remains **120 hours per team member** (the course workload expectation). Because the feature scope has

been deliberately reduced, the hours not consumed by feature implementation are allocated to testing, quality assurance, the OWASP/GDPR/WAI checklists, and documentation, in line with the “fewer features, done well and well-tested” guidance.

ID	Feature / Responsibility	Use Case	Priority	Category	Version	Effort	Lead
FR-01	User registration and login	UC-01	High	Core	V1	24h	Joan de la Fuente C.
FR-02	Secure WG creation & time-limited invitation links	UC-01	High	Core	V1	32h	Dzovani Koller
FR-03	Task creation and assignment (assignment, recurrence, undo completion)	UC-02	High	Core	V2	24h	Joan de la Fuente C.
FR-04	Cleaning schedule management (filter by status, recurring schedule)	UC-02	High	Core	V2	32h	Joan de la Fuente C.
FR-05	Shared expense tracking	UC-03	High	Core	V2	48h	Cemil Aslan
FR-06	Payment status overview & deterministic settlement	UC-03	High	Core	V3	32h	Cemil Aslan
FR-07	Shopping list management (mark/unmark bought, structured catalogue)	UC-04	High	Core	V2	32h	Daniel Petriceanu
FR-08	Targeted shopping request collection (mark/unmark bought)	UC-04	Med	Core	V2	24h	Daniel Petriceanu
FR-09	Aggregated household shopping list & shopping history	UC-04	Med	Core	V3	40h	Taras Trishch
FR-10	Shopping item suggestion/voting	UC-04	Med	Extended	V4	24h	Daniel Petriceanu
FR-11	Notifications and reminders	UC-05	Med	Extended	V4	32h	Daniel Petriceanu
FR-12	Dashboard with current responsibilities	UC-05	High	Core	V3	40h	Dzovani Koller
FR-13	History of tasks and payments	UC-05	High	Core	V3	32h	Andrei Ciuta
FR-14	One-click expense generation	UC-03	Med	Extended	V4	32h	Cemil Aslan
FR-15	Fair chore balancing via history	UC-02	Low	Extended	V5	32h	Andrei Ciuta
FR-16	Statistical history view (pie charts of costs/food/electricity, top buyer, expense development)	UC-05	Med	Core	V4	40h	Taras Trishch
FR-17	Swipe-based settlement approval with test account	UC-03	Low	Extended	V5	28h	Cemil Aslan
FR-18	QR-code scanner for one-click shopping item entry	UC-04	Low	Extended	V5	24h	Daniel Petriceanu
Feature implementation subtotal						**572h**	

Table 4: Iceberg List of Features, Use Case Mappings, and Estimation. Remaining hours within the 120h/person course budget are dedicated to testing, QA, security/GDPR/usability checklists, and documentation.

5 Domain Model

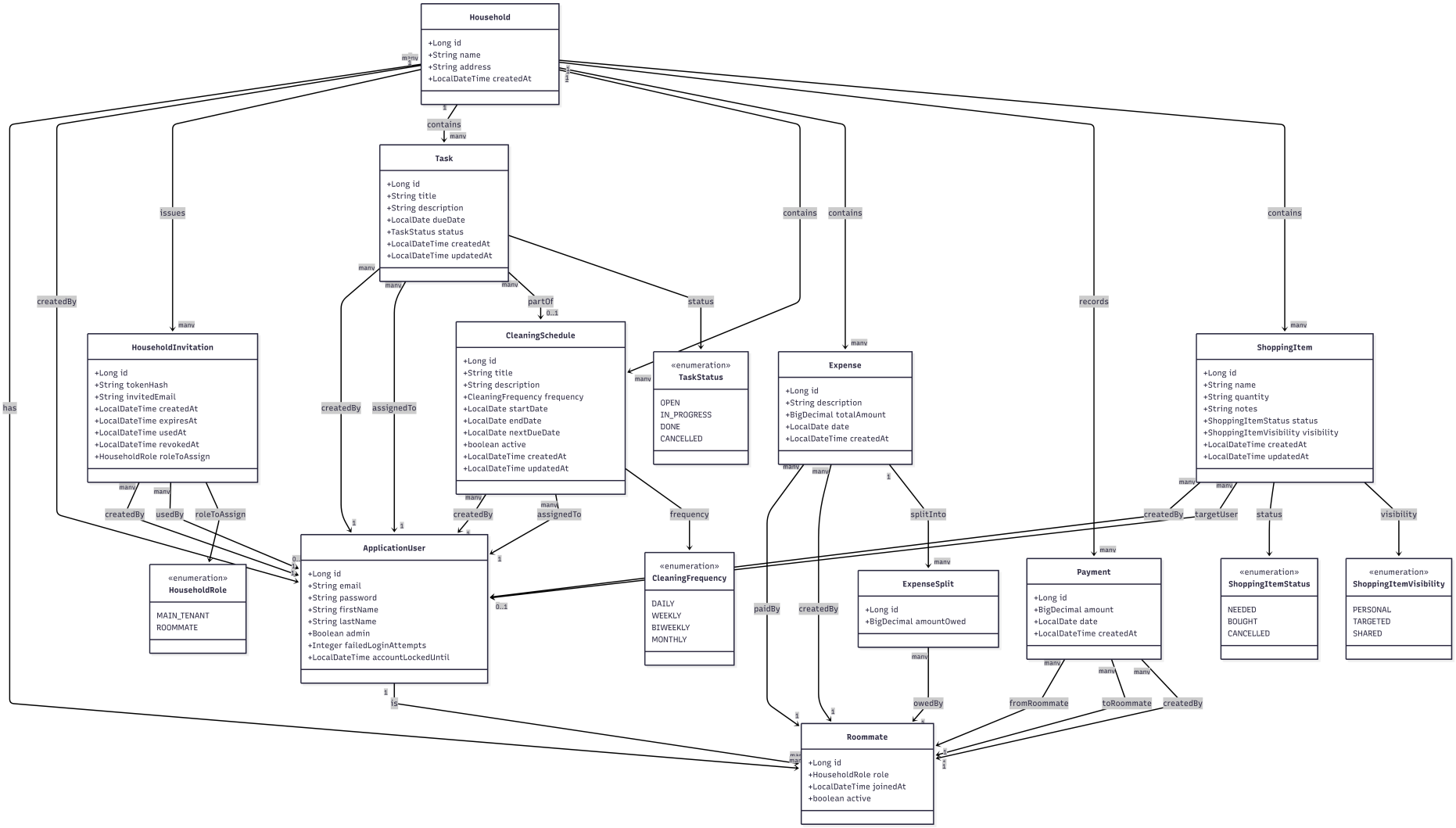


Figure 1: Detailed domain model in landscape format.

5.1 Domain Model Explanation

The domain model describes the main data structure of the WG Management System. The central entity is `Household`, which represents one shared flat and connects the main areas: roommates, tasks, cleaning schedules, expenses, and shopping.

A `Roommate` links an `ApplicationUser` to a `Household` with a role (`MAIN_TENANT` or `ROOMMATE`). Membership is established through a `HouseholdInvitation`: a single-use, time-limited, securely hashed token.

A `Task` stores a title, description, due date, and status (`TaskStatus`) and is assigned to one user; recurring chores are driven by a `CleaningSchedule` with a recurrence frequency (`CleaningFrequency`), and a task may optionally belong to one such schedule. A completion can be undone in case of mistakes.

The finance area consists of `Expense`, `ExpenseSplit`, and `Payment`. An expense records who paid and is divided into per-roommate splits (`amountOwed`); a `Payment` records a settling transfer from one roommate to another. Balances and the pending-versus-settled overview are computed deterministically by netting recorded payments against the splits. There is no per-split `settled` flag; settlement status is derived rather than stored.

Shopping is represented by a single `ShoppingItem` entity with a status (`NEEDED`, `BOUGHT`, `CANCELLED`) and a visibility (`PERSONAL`, `TARGETED`, `SHARED`) that can target a specific user. Items can be marked and unmarked as bought and form the household's shopping history.

Some features, such as the dashboard, the history view, the statistical history visualizations, fair chore balancing, and one-click expense generation, are modeled as derived behaviour instead of separate entities because they are calculated from existing domain data.

6 Working Structure & Project Plan

6.1 Distribution of Roles

The project work is distributed according to the primary roles assigned in the team structure. Each team member is mainly responsible for tasks that match their role while still supporting other areas when necessary.

- **Team Coordinator:** Organizes project planning, milestone tracking, and task distribution.
- **Technical Architect:** Designs the system architecture and supports backend/frontend implementation decisions.
- **UI/UX:** Designs user interfaces and ensures usability across the application.
- **Documentation Lead:** Maintains project documentation and prepares submission materials.
- **DevOps:** Supports deployment processes and technical infrastructure tasks.
- **Test Lead:** Responsible for testing activities and quality assurance.

6.2 Horizontal Responsibilities

In addition to their primary feature responsibilities, team members are responsible for several cross-functional tasks throughout the project lifecycle.

- **Project Coordination (Cemil Aslan):** Organizes meetings, tracks milestone progress, manages task distribution, and ensures deadlines are met.
- **Architecture Oversight (Dzovani Koller):** Maintains consistency between frontend, backend, and database architecture decisions.
- **UI/UX Consistency (Joan de la Fuente Camps):** Ensures a consistent user interface design and usability across all application modules and reference devices.
- **Documentation Management (Daniel Petriceanu):** Maintains project documentation, reports, diagrams, and submission materials, including user documentation.
- **Testing and Quality Assurance (Andrei Ciuta):** Oversees testing processes, bug tracking, the security/GDPR/usability checklists, and verifies feature stability before merges.
- **Deployment and Technical Support (Taras Trishch):** Monitors deployment functionality, resolves infrastructure-related issues, and supports integration processes.

6.3 Project Phases & Milestones

Timeline	Milestone	Deliverables
Week 1–2	**MS1 / V1: Project Setup Preparation**	Initial group meeting, introduction to the tutor, and discussion and selection of the project topic. Preparation and delivery of the PV presentation, as well as preparation and delivery of the PA presentation. Setup of GitLab and introduction of time-tracking structures.
Week 3–4	**MS2 / V2: Project Setup**	Fully configured project setup with working user registration/login and successful user-to-WG connection. Backend, frontend, H2 database, CI skeleton, and initial API documentation are available, together with initial work on core household management systems.
Week 5–6	**MS3 / V3: Core Household Management and Feature Expansion**	Working system for managing tasks, cleaning responsibilities, expenses, and shopping requests. Main Tenant and Roommate roles are enforced on the backend. Complete core feature set with dashboards, payment status overview, history views, and improved user visibility. Core workflows are tested on the defined reference devices.
Week 7–9	**MS4 / V4: Statistical Views & Selected Extensions**	Statistical history view, shopping suggestions/voting, one-click expense generation, and notifications. Extended features should not destabilize the core workflow.
Week 9–11	**MS5 / V5: Finalization**	Final version of the application with planned core features implemented, tested, and documented. Extended features such as the swipe-based settlement approval (with test account), the QR-code scanner for one-click shopping item entry, and fair chore balancing are delivered as implemented features, prototypes, or documented future work depending on remaining project capacity.

6.4 Project Boundaries (Out-of-Scope)

- **No Native App:** The project will only provide a responsive web application and not separate Android or iOS applications.
- **No Real Money Transfers:** The system tracks expenses, balances, and payment status, but does not handle actual money transfers or banking integrations.

- **No Full Accounting System:** WGMS does not provide tax reports, invoices, legally binding financial statements, or advanced bookkeeping features.
- **No NFC Features:** NFC-based appliance logging is no longer part of the scope; a QR-code scanner for shopping items is planned instead as a later extended feature.
- **No Voice or LLM Automation:** Voice-assisted input, speech transcription, and LLM-based natural-language command parsing are out of scope.
- **No Cross-WG Communication or Events:** Cross-WG chat, event planning, public feeds, and following other WGs are out of scope.
- **No Stock Prediction:** The dashboard provides a statistical history view rather than predictive stock forecasting.
- **No Legal Rental Management:** WGMS supports household organization but does not replace rental contracts, landlord communication, or legal tenant agreements.
- **No Smart Home Control:** The system does not control household devices or connect to IoT platforms.
- **No Universal Device Optimization:** The UI is optimized for the defined reference devices and viewport sizes. Other modern devices are supported on a best-effort basis, but the project does not guarantee pixel-perfect layouts for all smartphones, tablets, foldables, or desktop resolutions.
- **No commercial monetisation infrastructure:** payment processing for premium subscriptions, ad serving, micro-fee collection, B2B billing portals, and sponsorship marketplaces are out of scope. The architecture is designed to be compatible with the monetisation paths described in Section 2.1 but the implementation produces no commercial deliverable.

7 Supply Components (Deliverables)

The delivery is split into core deliverables and optional extended deliverables so that grading can focus on a stable core workflow while still allowing advanced features.

Core deliverables are the Angular web application, Spring Boot REST API, H2 persistence layer, authentication and authorization, WG membership, task and cleaning schedule management, expense and settlement module, shopping module with shopping history, dashboard with statistical history view, CI/CD configuration, deployment/setup documentation, API documentation, **user documentation** (end-user guide covering all core workflows), C4 diagrams, UML model, and test documentation.

Extended deliverables are the swipe-based settlement-approval interface (with a test account), shopping item suggestions/voting, one-click expense generation, fair chore balancing, and a QR-code scanner for one-click shopping item entry. These are implemented after the core system is stable and may be delivered as working features, prototypes, or documented future work.

The delivery explicitly excludes native mobile apps, real payment processing, smart-home control, NFC features, voice/LLM automation, cross-WG communication and events, stock prediction, legal rental management, and formal accounting.

8 Risk Assessment and Quality Management

8.1 Risk Matrix

The following risk matrix identifies the most important technical, organizational, legal, and adoption-related risks for the WGMS project. Each risk is assessed according to its probability and impact. Countermeasures are defined to reduce either the probability of the risk occurring or the damage caused if the risk occurs.

ID	Risk	Probability	Impact	Countermeasure
R-01	Low adoption by roommates because users continue using WhatsApp or informal communication instead of WGMS.	Medium	High	Keep the main workflows faster than using a chat message. Focus on dashboard visibility, reminders, and simple input forms. Introduce the system through the Main Tenant as the central sponsor.
R-02	Scope creep through additional features beyond the agreed core.	Medium	High	Apply strict prioritization. Core features are prioritized before the limited set of extended features. Extended features may be reduced to prototypes or documented as future work, keeping the stable core workflow clear for assessment.
R-03	Security or privacy breach exposing personal data or financial household information.	Medium	Critical	Use secure authentication, password hashing, HTTPS in deployment, RBAC, input validation, and least-privilege access. Verify against the OWASP ASVS checklist. Avoid logging sensitive user content or tokens.
R-04	Incorrect expense calculation or settlement logic leading to wrong balances between roommates.	Medium	Critical	Implement the settlement engine separately from the UI. Use decimal data types instead of floating-point arithmetic. Create unit tests for equal splits, unequal splits, rounding, deleted users, recomputation after expense edits, and netting of recorded payments.
R-05	GDPR compliance gaps because the system processes personal data such as names, emails, tasks, expenses, and shopping history.	Medium	High	Maintain a data inventory, define clear purposes for each data type, minimize collected data, provide deletion or anonymization options, and verify against the GDPR checklist.

R-06	Integration problems between the Angular frontend, Spring Boot backend, and H2 database.	Medium	High	Define REST contracts early, use consistent DTOs, provide sample data, and run integration tests in the CI/CD pipeline. Use a containerized local setup where applicable.
R-07	Data loss due to faulty database initialization, accidental deletion, or development mistakes.	Low	Critical	Version-control database initialization files, use test data for development, define an export strategy for demo data, and test restore procedures before final delivery.
R-08	Slow or cluttered views when dashboards or histories contain many records.	Medium	Medium	Use pagination, database indexes for frequently queried fields, and avoid loading all historical data at once.
R-09	Team availability risk due to uneven workload, illness, exams, or unclear responsibilities.	Medium	High	Use primary and backup roles, weekly progress checks, visible task boards, and early escalation of blockers. Critical features should not depend on only one team member.
R-10	External hosting restrictions or vendor lock-in.	Medium	Medium	Prefer university-provided or EU/EEA-based hosting where possible. Keep the application containerized so it can be moved between environments. Document all external services.
R-11	Notification or reminder features create unwanted spam or social pressure between roommates.	Low	Medium	Allow users to configure notification preferences. Keep reminders factual and task-based. Avoid public exposure of private WG information by default.
R-12	Optimization chaos caused by trying to support too many devices, browsers, and screen sizes.	Medium	High	Define a strict compatibility matrix. Optimize primarily for the defined reference platforms. Other devices are supported only on a best-effort basis.
R-13	The planned QR-scanning shopping workflow behaves inconsistently across devices/cameras.	Medium	Low	Treat QR scanning as a later extended feature with a manual entry fallback, validate scanned payloads on the backend, and test the workflow on the reference devices.

8.2 Quality Assurance Strategy

Quality assurance is integrated into the development process rather than treated as a final project phase. The main goal is to detect defects early, especially in security-related workflows, data consistency, and expense calculation.

Testing levels:

- **Unit tests:** Used for backend services, especially expense splitting, payment settlement, task assignment, and invitation handling.
- **Repository / database tests:** Used for critical repository queries to verify correct data retrieval, household scoping, and persistence behaviour. Every feature branch must include repository tests alongside service and endpoint tests.
- **Integration tests:** Used to verify the interaction between the REST API, database, authentication layer, and important business workflows.
- **Frontend tests:** Used for important UI workflows such as registration, adding expenses, completing tasks, viewing the dashboard, and (where implemented) the QR shopping flow.
- **Manual acceptance tests:** Used before each milestone to check whether the implemented features solve the intended WG use cases.
- **Reference-device tests:** Used to verify that core workflows remain usable on iPhone 12 Pro/iOS 26.x/Safari, Google Pixel 7/Android 16/Chrome 147.x+, and desktop browsers.

Metric-based checklists: Quality is measured against concrete, external metrics rather than ad-hoc judgement: the **OWASP ASVS** for security (NFR-01), the **GDPR checklist** for data protection (NFR-02), and the **W3C WAI** guidance and/or **Lighthouse** for usability and accessibility (NFR-04). The Test Lead (Andrei Ciuta) maintains and applies these checklists at each milestone.

Review process:

- Every major feature should be implemented on a separate Git branch.
- A feature should only be merged after another team member has reviewed the code.
- Critical logic, especially authentication and settlement calculation, should receive additional review by the QA/Risk responsible person or the technical architect.
- The CI/CD pipeline should run automated tests before code is merged into the main branch.
- Only features that are merged to the `dev` or `main` branch are considered for grading at milestone reviews.

Acceptance criteria: A feature is considered complete only if it is implemented, tested, reviewed, documented where necessary, and integrated into the current system version without breaking existing functionality.

8.3 Information Process

The team follows a simple communication and documentation workflow to ensure smooth coordination during development.

Communication:

- WhatsApp is used for daily communication and quick questions.
- Weekly meetings are held to review progress, discuss blockers, and assign new tasks.

- Additional meetings are scheduled when urgent issues arise.

Development Workflow:

- GitLab is used for source code management with stable code on `main` and integration on `dev`.
- Each team member works on feature branches named `feature/FR-ID-short-name`; bug fixes use `bugfix/short-description`.
- Features are reviewed, tested, and merged only after the CI/CD pipeline passes.
- Integration happens continuously throughout the project, but extended features should not destabilize the core workflow.

Documentation:

- Shared documents are used for requirements, diagrams, meeting notes, and reports.
- Documentation is updated whenever major changes are made.
- Known limitations of optional features should be documented clearly before final submission.

9 Legal, Compliance and External Hosting Constraints

WGMS processes personal data because users create accounts, join households, receive tasks, record expenses, and maintain a shopping history. Therefore, even though the project is developed in a university context, privacy and data protection requirements must be considered from the beginning.

9.1 Processed Data Categories

The system may process the following categories of data:

- **Account data:** Name, email address, password hash, authentication tokens, and user role.
- **WG membership data:** Household membership, invitation links, user roles such as Main Tenant or Roommate, and join/leave timestamps.
- **Task data:** Assigned chores, completion status, due dates, and task history.
- **Expense data:** Shared expenses, paid amounts, participants, per-roommate splits, recorded payments between roommates, and computed balances.
- **Shopping data:** Shopping items (with status and visibility), shopping history, and generated expense references.
- **Technical data:** Application logs, error messages, timestamps, and security-relevant events.

The project does **not** require processing of special categories of personal data such as health data, religious beliefs, political opinions, or biometric data. The system also does **not** execute real money transfers and does not need to store bank account numbers, credit card data, or payment provider credentials.

9.2 GDPR-Oriented Design Principles

The design of WGMS follows the most important GDPR-oriented principles, verified against the GDPR checklist referenced in NFR-02:

- **Purpose limitation:** Data is collected only for clear WG management purposes such as organizing chores, expenses, and shopping.
- **Data minimization:** Only data needed for the planned features is stored.
- **Transparency:** Users should be informed which data is stored and why. A short privacy notice should be included in the final documentation or application.
- **Storage limitation:** Data should not be kept longer than necessary. Invitation tokens should expire automatically. Deleted accounts should be removed or anonymized where possible.
- **Integrity and confidentiality:** Personal data must be protected through authentication, authorization, secure password handling, HTTPS in deployment, restricted database access, and careful log handling.
- **Accountability:** The team documents the main data categories, security measures, external services, and known privacy risks.

9.3 User Rights and Data Handling

The system should support basic user rights as far as possible within the project scope:

- **Access:** A user should be able to view their account information, WG membership, assigned tasks, expenses, and shopping history where relevant.
- **Correction:** Incorrect profile, household, task, shopping, or expense data should be editable by the user or by an authorized WG administrator.
- **Deletion:** Users should be able to leave a WG and request account deletion. Where financial history must remain understandable for other roommates, the user can be anonymized instead of fully deleting historical expense records.
- **Export:** If feasible, the system should provide a simple export of user-related data or allow manual database export during the project demo.

9.4 External Hosting Constraints

If WGMS is deployed externally, the following constraints apply:

- Hosting should preferably take place on university-provided infrastructure or servers located in the EU/EEA.
- If a third-party hosting provider is used, the provider should be documented as an external processor.
- If personal data is transferred outside the EU/EEA, the team should check whether appropriate safeguards such as adequacy decisions or standard contractual clauses are required.
- Production secrets, database passwords, and JWT secrets must not be committed to Git.
- Test environments should use artificial demo data instead of real personal data whenever possible.
- Application logs should not contain passwords, tokens, private data, or full financial details.

9.5 Security Constraints

Security is especially important because the system combines personal data, financial household records, and role-based WG administration. Therefore, the following constraints are defined and verified against the OWASP ASVS checklist (NFR-01):

- Passwords must be hashed and salted.
- Authentication tokens must expire after a reasonable time.
- Invitation links must be time-limited and difficult to guess.
- Users may only access data belonging to their own WG.
- Main Tenant or administrator functions must be protected by role checks on the backend, not only hidden in the frontend.
- If the QR-code scanner is implemented, scanned payloads must be validated on the backend before any shopping item is created.

9.6 Residual Compliance Risk

For the current project scope, no real payment execution, no banking integration, no voice or LLM processing, no cross-WG data sharing, and no special-category personal data are planned. The data the system stores is limited to account, household, task, expense, and

shopping information needed for WG management. This keeps the compliance risk manageable and appropriate for a university software engineering project.